

Manual App Base v6 - English

Manual: My Personal AI App

Build it with Claude as your interactive guide

📖 How to use this manual with Claude.ai

🗨️ What you'll build and why

📖 Mini-glossary (in case some words sound foreign)

🛠️ Why these specific tools

🔗 The Claude workflow: chat + Code + Tester

💡 Code philosophy: small, readable, separated

📖 Choosing the Gemini model

✓ Phase 0 — Prerequisites (install before starting)

📖 Phase 1 — Up-front decisions (5 min)

📖 Phase 2 — Local structure and Git (10 min)

🌐 Phase 3 — Private GitHub repo (5 min)

🔥 Phase 4 — Firebase project + Blaze upgrade (10 min)

🌐 Phase 5 — Multi-site Hosting: web + admin (3 min)

⚙️ Phase 6 — Local Firebase CLI + multi-site config (10 min)

💻 Phase 7 — Web frontend (Vite + React) (20 min)

📖 Phase 8 — Firestore data structure (reference)

🛡️ Phase 9 — Security Rules (15 min, critical)

📖 Phase 10 — Cloud Function: chat with Gemini (25 min)

📖 Phase 11 — Frontend persistent chat (25 min)

✂️ Phase 12 — Admin frontend (15 min)

👤 Phase 13 — Add the first admin (1 min, only first time)

🔗 Phase 14 — Build and deploy (10 min)

📖 Phase 15 — `CLAUDE.md` for Claude Code

📖 Phase 16 — Final commit v6.000

✓ Phase 17 — Final checklist

🔧 Common issues & solutions

🕒 Next steps

📖 Additional resources

Manual: My Personal AI App

Build it with Claude as your interactive guide

Stack: Firebase + Gemini + Google Auth + Code Tester

Manual version: v6.000

Systems: Windows 10/11 · macOS 13+

Built for: small teams, family circles, personal use, internal company tools — anyone who wants their own AI app without making it public.

Designed to be used with: Claude.ai (Pro plan or higher) as your interactive build agent.

📖 How to use this manual with Claude.ai

This is the most important section. Read it before anything else.

This manual is designed to be **uploaded to Claude.ai** as a PDF. Once uploaded, Claude reads the entire manual and becomes your **interactive build agent**. You don't need to read the manual end-to-end alone — you and Claude work through it together, phase by phase.

Step 1 — Open Claude.ai with a paid plan

Go to `claude.ai` and log in. We recommend a **Pro plan** (\$20/month at the time of writing) or higher. The free plan works but is heavily rate-limited; you may run out of messages mid-build.

Step 2 — Start a new conversation and upload this PDF

1. Click **“New chat”** or **“Start new conversation”**.
2. Click the **paperclip icon** at the bottom (file upload).
3. Select this PDF (`Manual_App_Base_v6_english.pdf`).
4. Wait a few seconds for Claude to read it.

Step 3 — Use this exact starter prompt

Copy and paste this into the chat:

```
I have just uploaded a manual for building a personal AI app on Firebase
+ Gemini. I want you to be my interactive guide.

Here is what I need from you:

1. Walk me through the manual phase by phase, in order. Don't dump
   everything at once. Wait for me to confirm each phase is done before
   moving on.

2. At each phase, tell me exactly what to do, paste exact commands,
   and explain what each command does in plain language. Adapt to my
   operating system (I'll tell you Windows or macOS).

3. When I need to create a file with code, generate the complete file
   contents inside an artifact so I can copy or download it.

4. If I paste an error message, diagnose it precisely and give me the
   fix. Don't guess.

5. Assume I have minimal technical experience. Be patient. If I ask
   "what does X mean?", explain it simply with an analogy if helpful.

6. Always remind me which phase we're on at the start of each response.

I am on [Windows / macOS]. My desired project name is [my-app] (or tell
me to pick one). Let's start with Phase 0.
```

Replace `[Windows / macOS]` and `[my-app]` with your actual choices, then send.

Step 4 — Work phase by phase

From here, just follow Claude's lead. Some tips:

- **If you don't understand something**, say *“Explain that in simpler terms”* or *“Why am I doing this?”*. Claude will adapt.
- **If a command fails**, paste the **exact error message** to Claude. Don't summarize. Claude needs the raw text to diagnose.
- **If you want to skip ahead**, you can. Tell Claude *“I've already done Phase 1 to 3, let's start at Phase 4”*.
- **If you take a break**, when you come back, say *“I'm back, last time we finished Phase X. Let's continue.”* Claude will pick up where you left off.
- **If you make a mistake**, say *“I think I broke something at Phase X, can you help me undo and retry?”*. Claude will guide recovery.

Step 5 — Pair Claude.ai with Claude Code (recommended for fastest results)

For maximum efficiency, install **Claude Code** (the terminal tool) too. See Phase 0 for instructions. The reason: Claude.ai (chat) is great for thinking and writing code, but executing commands on your computer eats messages fast. Claude Code lives in your terminal and executes commands very efficiently. The combination of both is what makes this manual fit comfortably within a Pro plan's message budget.

We dedicate an entire section ("The Claude workflow") to explaining this division of labor.

What if I don't have a paid Claude plan?

You can still use this manual:

- **Read it directly as a PDF** and follow it manually (no Claude as guide).
- Use the free Claude tier sporadically to ask specific questions.
- Or use any other AI assistant capable of reading PDFs.

But this manual is **designed and optimized** for the Claude Pro experience.

What you'll build and why

This manual guides you, step by step, to build **a personal web application** that:

- Has **its own URL** like `https://my-app.web.app`, accessible from any phone, computer, or tablet.
- Requires **Google login** before letting anyone in.
- Only allows access to **users you authorize manually** (e.g., yourself, a colleague, two team members — and that's it).
- Includes an **AI chat** (Google's Gemini) with persistent memory — every conversation is saved, none is forgotten.
- Has an **admin zone** where you (and only you) can add or remove authorized users.
- Has a **Code Tester** always visible at the bottom-right, showing in real time what's happening inside the app (like an airplane's black box).
- Stores **all data in your own database** (not on OpenAI's disk, not anyone else's — it lives in your Google account).

How much will it cost

Two separate costs:

1. Cost of running the app (Firebase + Gemini)

- **€0 / \$0 during the first year** for personal/team use (you stay within free tiers).
- For high-volume usage (hundreds of conversations per day), you might reach €1-5 / month.
- We will set an automatic alarm so you never exceed €5 / \$5 per month without knowing.

2. Cost of building it with Claude

- **Pro plan** (~\$20/month): enough to build this entire app if you follow the workflow we describe.
- **Max 5x** (~\$100/month): plenty of headroom. You could build this app and many more.

- The trick to fit the Pro plan is **dividing the work between three tools** — exactly what we explain in the next major section.

How long will it take

- **First time:** 4-6 hours if you follow the manual closely with Claude as your guide.
- **If you have prior experience:** 1-2 hours.
- **You don't need to do it all at once.** Stop at any phase, come back another day, tell Claude where you left off.

What you need to know in advance

Nothing specific. This manual is designed for someone with no technical background to succeed. You only need:

- A **credit card** to activate Firebase Blaze (it stays free if you stay within normal usage — the card is just a “guarantee”).
- A **Google account**.
- A **Claude Pro plan** (recommended — see “How to use this manual with Claude.ai” above).
- The ability to **copy and paste text** and **follow instructions in order**.

Mini-glossary (in case some words sound foreign)

Term	What it means
Frontend	The visible part of the app: what you see in the browser. Buttons, text, forms.
Backend	The invisible part: code that runs on servers. Talks to the database and to the AI.
Cloud Function	A small piece of code that lives on Google's servers and runs when called.
API key	A password services give you (like Gemini) so your code can talk to them.
Firestore	Firebase's database. Imagine an infinite real-time spreadsheet.
Auth	Short for "Authentication". The system that controls who's who (login, logout).
Repo / Repository	A special code folder stored on the internet (on GitHub) with full change history.
Deploy	Publishing your app to the internet so people can access it.
Token	A small chunk of text (about 4 characters) the AI uses as a unit. 1 million tokens $\approx$ 1,500 pages.
Context	The conversation memory the AI has present in each response.
PowerShell / Terminal	Programs to type commands into your computer instead of clicking with the mouse. Windows = PowerShell, Mac = Terminal.
Claude chat	claude.ai or the Claude app. Where you talk to Claude and get code/explanations.
Claude Code	An Anthropic terminal tool. Reads your project folder directly and executes commands.

🔧 Why these specific tools

Why Firebase (and not AWS, Vercel, Supabase...)

Firebase is Google's "all-inclusive" service for small apps:

- Google login is configured with 2 clicks.
- The database works in **real time** (when someone writes, others see it instantly — key for the chat and for the Code Tester).
- Hosting already provides a `.web.app` URL with HTTPS without doing anything.
- Everything is in one place: one console, one bill, one login.
- It is **free up to a substantial volume** (5 GB hosting, 1 GB database, 2M function invocations/month).

Alternatives: AWS (too complex for a personal app), Vercel (no native database), Supabase (good option, less integrated with Google Auth).

Why Gemini (and not GPT, Claude, Llama...)

Three concrete reasons:

1. **The largest context window on the market:** 1 million tokens (Gemini 2.5 Pro) or up to 2 million (Gemini 1.5 Pro via API). This means very long conversations without the AI "forgetting" early messages. For comparison: standard ChatGPT has 128,000 tokens; Claude has 1 million as well.

2. **Natively multimodal:** you can send images, short videos, audio... directly inside the same conversation, without converting anything.
3. **Price and integration ease** when already in the Google ecosystem: API key in 30 seconds, no separate billing account needed, integrates with Firebase frictionlessly.

Note: even though the chat *interface* of this app uses Gemini, you're still building it with Claude. Different tools for different jobs.

Why Vite + React

- **React:** the most popular framework for building interfaces. Thousands of examples online; Claude knows it perfectly.
- **Vite:** the tool that prepares React for the browser. Very fast (unlike the classic "Create React App" which takes ages).

Why private GitHub

Because your app's code may contain sensitive information (configs, personal comments) and you don't want it public. GitHub offers unlimited free private repositories.

🔗 The Claude workflow: chat + Code + Tester

This section is critical if you have a Claude Pro plan (or Max 5x, the cheapest of the Max tier). It explains how to build this app without running out of messages mid-way.

The problem: Claude plans have limits

All paid Claude plans have a **5-hour rolling window** that resets. When your quota runs out, you wait.

Plan	Approximate quota per 5h	Monthly cost
Free	very low	\$0
Pro	~45 messages	~\$20
Max 5x	~225 messages (5x Pro)	~\$100
Max 20x	~900 messages (20x Pro)	~\$200

These figures are user-reported estimates and can vary. Actual consumption depends on message length, attached files, model chosen (Opus consumes more than Sonnet or Haiku), etc.

If you tried to build this entire app **only via claude.ai chat**, every small operation would eat messages:

- Asking for a terminal command → 1 message
- Running it and pasting the output back → 1 more message
- Asking why it failed → 1 more
- Getting the corrected code → 1 more

Multiply by 100 operations and you've spent the day waiting for the quota to reset.

The solution: divide the work into three roles

Each kind of task has an optimal tool. Used correctly, **a complete app like this fits within a Pro plan's budget.**

Role 1: Claude chat (claude.ai) — the architect 🧠

What it does: designs, decides, writes new code from scratch.

When you use it:

- “Design the database structure for a chat app”
- “Generate the complete `Chat.jsx` component code”
- “Here’s an error I’m getting, what’s happening?”
- “Review this code and tell me if it does what we want”
- “Generate a zip with all the files needed for the first version”

What it consumes: messages from your Claude plan.

What it delivers: code (text in chat, or zip files via Artifacts).

Strength: thinks deeply, has lots of context, helps you decide.

Weakness: cannot see your disk. You have to explain everything. Mechanical execution tasks are expensive in messages.

Role 2: Claude Code (terminal) — the executor 🖥️

What it does: executes on your computer. Reads your project files directly, runs commands, builds, deploys, pushes to git, installs dependencies.

When you use it:

- “Unzip this archive into the project folder and update the files”
- “Build both frontends and deploy to Firebase”
- “Check that all 7 version locations are in sync and fix them”
- “Commit and push with this message”
- “Install firebase-tools globally and log in”

What it consumes: also from the same Claude plan (counts toward the same limit), but **far more efficient** for execution because:

- It has direct access to your files (no need to explain what’s there).
- It can run terminal commands directly (no copy-paste needed).
- A single Claude Code session can do 10-20 operations with very little “explanatory context”.
- It uses tools (`read` , `write` , `bash`) that are cheaper in tokens than normal conversation.

What it delivers: real disk changes, deploys, GitHub commits.

Strength: fast, efficient, doesn’t waste time explaining.

Weakness: not creative. **Don’t use it to design new things.** If you ask it “create me a new component”, it will improvise. The golden rule: **chat decides, Code executes.**

Role 3: Code Tester — the eyes 👁️

What it does: reads `testerLogs/` from Firestore in real time and displays them as a green floating console on the screen.

When you use it: whenever your app is open. It’s passive — it just watches.

What it consumes: nothing from your Claude plan. Zero. Only Firestore reads (which fall within Google’s free tier).

What it delivers: immediate feedback about what’s happening internally, without having to ask anyone.

Strength: free, real-time, always available.

Weakness: only shows what you've programmed it to log (which is why it's important to call `logTester(...)` from the backend at all the right places).

The combined advantage

When all three roles work together:

- **Chat messages** are reserved for **real thinking**: architecture, decisions, new code, complex debugging.
- **Mechanical execution** drops to Claude Code, where one session does many operations at low cost.
- **Feedback** comes from the Code Tester, free and live — no need to ask Claude “what happened?” when you can see it yourself.

Practical result: **on a Pro plan you can build this entire app without exhausting the limit**, because we reserve expensive messages for important decisions.

A concrete example of a change cycle

Suppose you want to add a new feature (e.g., export the chat to PDF):

1. **Chat (claude.ai)** — 3-5 messages:
 - “I want to add PDF export to the chat. What’s the architecture?”
 - You discuss options, decide on a library.
 - “Generate the complete `ExportPDF.jsx` component and the modifications to `Chat.jsx` to integrate it.”
 - You receive a zip.
2. **Claude Code** — 1 session, 5-10 internal operations (low consumption):
 - “Unzip this into the project folder, update existing files, run `npm install jspdf` in `web/`, build, deploy to hosting:web.”
 - Done without further questions.
3. **Code Tester** — free:
 - You open the app, test the export button.
 - You see `[export-pdf]` logs appearing in the Code Tester.
 - If it fails, you see the error directly. If it works, you see the result.

Total Pro plan consumption: 4-6 messages for an entire feature. That’s how the Pro plan lasts you all day.

💡 Code philosophy: small, readable, separated

This philosophy is a **direct consequence** of the workflow above. With well-divided code, Claude Code can make changes without touching what shouldn’t be touched, and Claude chat can think about specific pieces without needing to understand the whole app.

The golden rule

One file, one feature. Never a monolithic `App.jsx` with 2,000 lines that no one understands. Each piece of the code lives in its own file:

File	Single responsibility
<code>Chat.jsx</code>	Only the chat with the AI
<code>CodeTester.jsx</code>	Only the live logs
<code>AdminPanel.jsx</code>	Only administration
<code>useAuth.js</code>	Only authentication
<code>firebase.js</code>	Only Firebase configuration
<code>constants.js</code>	Only constants (version, name, etc.)

If you want to add a new feature (e.g., a calendar), you create `Calendar.jsx`. **You do not put it inside `Chat.jsx` or `App.jsx`.**

Why this matters so much

1. **Any human can read the code line by line, function by function**, and understand what it does. Without getting lost in 2,000 lines.
2. **Errors localize quickly**: if the chat fails, you know the problem is in `Chat.jsx` or in the Cloud Function. Nowhere else.
3. **Claude Code can modify only one piece** without touching the rest. If you give it a monster, it returns a bigger monster.
4. **You save messages on the Pro plan**: if Claude has to “understand” 2,000 lines every time, it spends more tokens. If it has to understand 100, it spends fewer.
5. **Your future self in 6 months will thank you**: when you return to the app after a break, you want things clearly organized.

The Code Tester as guardian of the philosophy

The **Code Tester** (the black box with green text at the bottom-right) is **not just a debugging tool**. It is the visible consequence of this philosophy:

- Every important piece of code that does something leaves a trace (a log) with its `type` and `message`.
- When you open the Code Tester and see `[gemini-call]`, you know the Gemini module did something.
- When you see `[error]`, you know exactly where to look.
- When you see many logs from one module and none from others, you know something is poorly distributed.

It’s a thermometer of your app’s architectural health. If the Code Tester fills with incoherent logs or never shows anything, your code is becoming a monster.

When working with Claude Code (and you should)

Always insist that it create new files when adding features, not inflate existing ones. Magic phrase:

“Add this feature as a new component in its own file. Do not modify `App.jsx` except to import the new component.”

If Claude Code starts dumping 200 lines into `App.jsx`, **stop it**. Ask it to extract that into a separate file. The app grows well only if it stays modular.

Choosing the Gemini model

Quick comparison

Model	Context	Speed	Multimodal	Price (relative)	Best for
Gemini 2.5 Pro	1M tokens	Medium	Yes (text, images, audio, video)	\$\$	Recommended default. Big context, very good quality.
Gemini 2.5 Flash	1M tokens	Fast	Yes	\$	When you want quick responses and don't need maximum quality.
Gemini 3 Pro	1M tokens	Medium	Yes	\$\$\$	The smartest but the most expensive. Reserved for complex tasks.
Gemini 3 Flash	1M tokens	Fast	Yes	\$\$	Quality/speed balance with newer technology.
Gemini 1.5 Pro	2M tokens	Slow	Yes	\$\$\$\$	Only if you need more than 1M tokens (rare).

What we recommend in this manual

We start with **Gemini 2.5 Pro** because it offers the best balance:

- Massive memory (1 million tokens $\approx$ 1,500 pages of conversation fit in the AI's "short-term memory").
- Native image support.
- Reasonable price.
- Available to anyone with a free API key.

Changing it later is changing 1 line of code. If it feels slow or expensive, switch to Flash. If you want maximum power, switch to 3 Pro. We'll mark the exact line later.

What about images?

Good news: **Gemini models are natively multimodal.** The same model that responds in text also "sees" images. You don't need to switch models.

In Phase 10 (Cloud Function) and Phase 11 (frontend) we provide **two versions of the code**:

- **"Text only"** version (to start simple).
- **"Text + images"** version (when you want users to upload photos to the chat).

Use cases for image support:

- "What does this label say?" (product photo)
 - "What's in this picture?" (description)
 - "Translate this sign" (foreign-language text)
 - "What recipes can I make from these ingredients?" (fridge photo)
 - "Diagnose this error" (error screenshot)
-

✓ Phase 0 — Prerequisites (install before starting)

📎 **If working with Claude.ai as guide:** tell Claude “I’m starting Phase 0. I’m on [Windows / macOS]. Walk me through prerequisite installation.” Claude will adapt instructions to your OS and confirm each step.

Common tools (both platforms)

Tool	Minimum version	Purpose
Node.js	20.x LTS	Language that runs the backend code and build tools
npm	10.x (ships with Node)	Package manager — installs the libraries we’ll use
Git	2.40+	Version control — keeps the code change history
GitHub CLI (gh)	2.x	Create the private repo from terminal with one command (optional but convenient)
VSCode	latest	Code editor (recommended — free, easy)
Claude Code	latest	The executor — install and activate with your Pro plan

Windows — step-by-step installation

1. Node.js:

- Go to nodejs.org.
- Download the **LTS** version (left button, usually 20.x).
- Open the downloaded `.msi` → Next, Next, accept license, Next.
- **Check “Automatically install necessary tools”** (important — installs extras we need).
- Wait for it to finish (5-10 minutes).

2. Git:

- Go to git-scm.com/download/win.
- Download and open the installer.
- Accept all defaults except:
 - “Choose the default editor used by Git” → choose **Visual Studio Code** (if installed) or notepad.
 - “Adjusting your PATH environment” → check **“Git from the command line and also from 3rd-party software”**.
 - “Configuring the line ending conversions” → check **“Checkout as-is, commit Unix-style line endings”** (better for Mac compatibility).

3. GitHub CLI:

- Open PowerShell (search “PowerShell” in start menu → right-click → “Run as Administrator”).
- Type:

```
winget install --id GitHub.cli
```

- If it asks for confirmation, type **Y** and Enter.

4. **VSCode** (recommended):

- code.visualstudio.com → Download for Windows → installer → Next, Next.

5. **PowerShell permission** (only needed once):

- Open regular PowerShell (not Admin).
- Type:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

- When prompted, type **Y** and Enter.
- **Why:** by default Windows blocks tools like **npm**. This line allows them only for your user.

6. **Claude Code:**

- In PowerShell:

```
npm install -g @anthropic-ai/claude-code
```

- Then from any project folder:

```
claude
```

- First time it asks for authentication. Browser opens. Log in with your Claude account (Pro or Max).

7. **Verify everything works:**

- Close PowerShell and reopen it (so it picks up changes).
- Type these commands one by one:

```
node --version
npm --version
git --version
gh --version
claude --version
```

- Each should respond with a version number.

macOS — step-by-step installation

1. **Homebrew** (program installer):

- Open Terminal (**Cmd + Space** → type “Terminal” → Enter).
- Paste this line and Enter:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- It will ask for your Mac user password. Type it (you won't see characters as you type — that's normal) and Enter.
- Wait 5-10 minutes.
- At the end it prints 2 lines starting with **(echo ...) >>** **~/ .zprofile**. **Copy and run them one by one** — without this, Homebrew won't work.

2. **Tools via Homebrew** (all at once):

```
brew install node@20 git gh
```

Wait 10-15 minutes.

3. **If you had an older Node version:**

```
brew unlink node  
brew link --overwrite node@20
```

4. **VSCode** (recommended):

```
brew install --cask visual-studio-code
```

5. **Claude Code:**

```
npm install -g @anthropic-ai/claude-code
```

Then from any folder:

```
claude
```

Authenticate via browser.

6. **Verify:**

- Close Terminal and reopen.
- Type:

```
node --version  
npm --version  
git --version  
gh --version  
claude --version
```

Accounts you'll need

- **GitHub:** github.com/signup (if you don't have one). Free.
- **Google:** the same account you use for Gmail.
- **Firebase:** log in at console.firebase.google.com with the Google account.
- **Google AI Studio:** aistudio.google.com with the same Google account.
- **Anthropic / Claude:** claude.ai — Pro plan or higher recommended.
- **Credit card:** for Firebase Blaze (you won't be charged if you stay within free tier).

📁 Phase 1 — Up-front decisions (5 min)

Write these decisions on paper or a note. You'll use them throughout.

Decision	Example	Where it's used	Changeable later?
Project short name	<code>my-app</code>	Folder, repo, Firebase ID	The Firebase ID can NEVER be changed. Choose carefully.
Friendly name	<code>My App</code>	Browser title, login page	Yes, easily.
Firebase Project ID	<code>my-app</code>	URL: <code>my-app.web.app</code>	NO. If taken globally, try <code>my-app-2026</code> , <code>my-app-personal</code> , etc.
Initial admin email	<code>admin-name@gmail.com</code>	First doc in the database	Yes, but it's the "egg" — be careful.
Authorized user emails	<code>admin-name@gmail.com</code> , <code>colleague@gmail.com</code>	Access whitelist	Yes, via the admin panel.
Firestore region	<code>eur3</code>	Multi-region Europe	NO. Choose <code>eur3</code> if in Europe.
Functions region	<code>europe-west1</code>	Belgium, low latency from Europe	Yes, but requires redeployment.

💡 **About the Project ID:** imagine you ask for `lasagna`. If anyone in the world has asked before, you can't have it. Choose a specific name. Example: if you're called Maria and work at Acme Corp, try `acme-maria-app`. Likely available.

💡 **About `eur3`:** the Europe multi-region. Your data lives in Belgium, the Netherlands, and Finland simultaneously (replicated). If you're in Europe, this is the best option for compliance (GDPR) and speed.

From here, the manual uses `my-app` as example. **Replace with your real name** when running commands.

🔗 Phase 2 — Local structure and Git (10 min)

📋 **Claude Code shortcut:** tell Claude Code "Create the initial structure for `my-app` according to the manual: `web/`, `admin/`, `functions/`, empty `firebase.json`, `.gitignore`, `README.md`. Run `git init`. Then create a private GitHub repo named `my-app` and set it as origin." It will do this entire phase in one session.

2.1 Create project directory

Windows (PowerShell)

```
cd $env:USERPROFILE
mkdir projects -Force
cd projects
mkdir my-app
cd my-app
```

macOS (Terminal)

```
mkdir -p ~/projects
cd ~/projects
mkdir my-app
cd ~/projects/my-app
```

2.2 Initialize Git

```
git init
git config user.name "Your Name"
git config user.email "your-email@gmail.com"
```

2.3 Create folder structure

Windows (PowerShell)

```
mkdir web
mkdir admin
mkdir functions
```

macOS (Terminal)

```
mkdir web admin functions
```

Final project structure will be:

```
my-app/
├── web/                                User frontend (Vite + React)
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── constants.js        Holds the version (1 of 7 places)
│   │   └── index.css
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
├── admin/                             Admin frontend (Vite + React)
│   └── (same as web/)
├── functions/                         Cloud Functions (Node.js 20)
│   ├── index.js
│   └── package.json
├── firebase.json                     Global Firebase config
├── .firebaserc                      Multi-site targets
├── firestore.rules                  Security rules
├── firestore.indexes.json           Indexes (empty initially)
├── .gitignore
├── README.md
└── CLAUDE.md                       Optional instructions for Claude Code
```

2.4 Create `.gitignore`

Create `.gitignore` (with leading dot) in `my-app/` :

```
node_modules/  
dist/  
build/  
.env  
.env.local  
.env.production  
.firebase/  
runtimeconfig.json  
*.log  
.DS_Store  
Thumbs.db  
.vscode/*  
!.vscode/settings.json  
.idea/
```

2.5 First commit

```
git add .gitignore  
git commit -m "v6.000 - Initial .gitignore"
```

🌐 Phase 3 — Private GitHub repo (5 min)

Option A — GitHub CLI (recommended)

```
gh auth login
```

Follow the flow:

1. **Where do you use GitHub?** → `GitHub.com`
2. **Preferred protocol?** → `HTTPS`
3. **Authenticate Git with your GitHub credentials?** → `y`
4. **How would you like to authenticate?** → `Login with a web browser`
5. One-time code → paste into browser → authorize.

Then:

```
gh repo create my-app --private --source=. --remote=origin --description  
"My personal AI app"
```

Option B — Via the web

1. Go to `github.com/new`.
2. Repository name: `my-app`.
3. Check **Private**.
4. **DO NOT** check "Initialize with README".
5. Click "Create repository".
6. Copy the HTTPS URL.
7. In terminal:

```
git remote add origin https://github.com/YOUR-USERNAME/my-app.git
```

Initial push (both options)

```
git branch -M main  
git push -u origin main
```

🔥 Phase 4 — Firebase project + Blaze upgrade (10 min)

📖 This phase is browser-based. Claude can guide you click by click but cannot do it for you (no CLI for Firebase project creation).

4.1 Create Firebase project

1. Go to `console.firebase.google.com` (log in with Google).
2. Click **“Add project”** (blue button).
3. **Project name:** `my-app`. If taken, try variants.
4. Note the “Project ID” — you’ll use it throughout.
5. Click **Continue**.
6. **Disable Google Analytics** → Create project.
7. Wait 30 seconds → **Continue**.

4.2 Upgrade to Blaze (pay-as-you-go)

This step enables AI and Cloud Functions.

1. Bottom-left → **“Spark Plan”** or **“Upgrade”** button.
2. Click **Upgrade** → **Blaze** → **Purchase**.
3. If no billing account → **“Create new account”** → enter card details.

💡 **Don’t worry about cost:** Blaze has a **generous free tier**:

- 2M Cloud Functions invocations/month
- 5 GB hosting
- 1 GB database
- 50,000 free Firestore reads/day

For real personal use, you’ll never pay. The card is just a “guarantee” so Google doesn’t grant unlimited resources to bots.

4.3 ⚠️ CONFIGURE BUDGET ALERT

This is the step that protects you from surprises. Don’t skip it.

1. Firebase Console → ⚙️ top-left → **Usage and billing**.
2. **Details & settings** tab → click **Set spending alerts**.
3. Opens the GCP console → Budgets & alerts.
4. Click **Create Budget**:
 - **Name:** `my-app-monthly`
 - **Time range:** Monthly
 - **Target amount:** **5 EUR / 5 USD** (or your preferred ceiling)
 - **Threshold rules:** 50%, 90%, 100%
 - **Email recipients:** your email
5. Click **Save**.

4.4 Activate services

In the Firebase Console left panel, activate:

Authentication

1. **Authentication** → **Get Started**.
2. Sign-in method → **Google** → Enable.
3. Project public-facing name: `My App`. Email: yours.
4. Save.

Firestore Database

1. **Firestore Database** → **Create database**.
2. **Production mode** → Next.
3. Location: `eur3` (`Europe-multi-region`) → Enable.

⚠ **Firestore region cannot be changed later.** Europe → `eur3` . US → `nam5` . When in doubt, `eur3` is a fine general option.

Hosting

1. **Hosting** → **Get Started** → Next, Next, Continue.

Cloud Functions

1. **Functions** → **Get Started**.

🌐 Phase 5 — Multi-site Hosting: web + admin (3 min)

The main app and admin zone live on separate domains. Benefits:

- Admin never loads for normal users.
- Independent deployment.
- No traces of admin code visible to others.

In Firebase Console → **Hosting**:

1. There's a default site `my-app` with URL `my-app.web.app` . Good.
2. Click **Add another site** → Site ID: `my-app-admin` → Add.

Result:

- `my-app.web.app` (for users)
- `my-app-admin.web.app` (for admins)

⚙️ Phase 6 — Local Firebase CLI + multi-site config (10 min)

6.1 Install Firebase CLI

```
npm install -g firebase-tools
```

💡 **Permission error on macOS:** `sudo npm install -g firebase-tools` . Asks for your Mac password.

💡 **Error on Windows:** reopen PowerShell as Administrator.

6.2 Login

```
firebase login
```

Opens browser. Choose your Firebase Google account. Confirm. Should say `Logged in as ...` .

6.3 Link directory to Firebase project

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app
firebase use --add
```

macOS (Terminal)

```
cd ~/projects/my-app
firebase use --add
```

Select `my-app` (your Project ID). Alias: `default` .

6.4 Create `firebase.json`

⚠ **Don't run `firebase init`** — it overwrites things and complicates. We create files by hand.

In `my-app/`, create `firebase.json` :

```
{
  "hosting": [
    {
      "target": "web",
      "public": "web/dist",
      "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
      "rewrites": [
        { "source": "**", "destination": "/index.html" }
      ],
      "headers": [
        {
          "source": "**/*.@(js|css)",
          "headers": [
            { "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }
          ]
        }
      ]
    },
    {
      "target": "admin",
      "public": "admin/dist",
      "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
      "rewrites": [
        { "source": "**", "destination": "/index.html" }
      ]
    }
  ],
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  },
  "functions": [
    {
      "source": "functions",
      "codebase": "default",
      "runtime": "nodejs20"
    }
  ]
}
```

6.5 `.firebaserc`

```
{
  "projects": {
    "default": "my-app"
  },
  "targets": {
    "my-app": {
      "hosting": {
        "web": ["my-app"],
        "admin": ["my-app-admin"]
      }
    }
  }
}
```

⚠ **Replace all 4 occurrences** of `my-app` with your real Project ID.

6.6 Empty initial rules

`firestore.indexes.json` :

```
{ "indexes": [], "fieldOverrides": [] }
```

`firestore.rules` (provisional, replaced in Phase 9):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

6.7 Test deploy

```
firebase deploy --only firestore:rules
```

Should say `+ Deploy complete!` . Your folder is now connected to Firebase.

📁 Phase 7 — Web frontend (Vite + React) (20 min)

📎 **Pro plan tip:** when finished with Phase 6, ask Claude.ai (chat):

“Generate a zip with the complete `web/` folder per the manual: `package.json`, `vite.config.js`, `src/main.jsx`, `src/App.jsx`, `src/components/Chat.jsx`, `src/components/CodeTester.jsx`, `src/hooks/useAuth.js`, `src/services/firebase.js`, `src/constants.js`, `index.html`.” You receive one zip. Then to Claude Code: *“Unzip into `web/` replacing existing files, then run `npm install`.”* Saves 20+ messages over creating files one by one.

7.1 Initialize Vite

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app\web
npm create vite@latest . -- --template react
```

macOS (Terminal)

```
cd ~/projects/my-app/web
npm create vite@latest . -- --template react
```

If asked “Current directory is not empty”, answer **Yes**.

7.2 Install dependencies

```
npm install
npm install firebase react-router-dom
```

7.3 web/src/constants.js

```
// web/src/constants.js
// Place 1 of 7 where the version lives. Sync the others on every bump.
export const VER = "v6000";
export const APP_NAME = "My App";
export const APP_TAGLINE = "AI chat — personal use";
```

7.4 web/src/services/firebase.js

Create folder `services/` and file:

```
// web/src/services/firebase.js
// _____
// Central Firebase configuration. Every other piece of
// the frontend imports from here.
// _____
import { initializeApp } from 'firebase/app';
import { getAuth, GoogleAuthProvider } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getFunctions } from 'firebase/functions';

// THESE VALUES are obtained in step 7.5
const firebaseConfig = {
  apiKey: "...",
  authDomain: "my-app.firebaseio.com",
  projectId: "my-app",
  storageBucket: "my-app.appspot.com",
  messagingSenderId: "...",
  appId: "..."
};

export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
export const functions = getFunctions(app, 'europe-west1');
export const googleProvider = new GoogleAuthProvider();

// Force account picker (useful if you have multiple Google accounts)
googleProvider.setCustomParameters({ prompt: 'select_account' });
```

7.5 Get Firebase credentials

1. Firebase Console → ⚙️ Project Settings.
2. **General** → **Your apps** → Add app → web `</>`.
3. App nickname: `web` . **DO NOT** check “Set up Firebase Hosting”.
4. Register app → copy the 6 `firebaseConfig` values → paste into `web/src/services/firebase.js` .

7.6 web/src/hooks/useAuth.js

```
// web/src/hooks/useAuth.js
// -----
// Hook that centralizes Google login.
// Also checks if the email is in the whitelist.
// -----
import { useState, useEffect } from 'react';
import { onAuthStateChanged, signInWithPopup, signOut } from
'firebase/auth';
import { doc, setDoc, getDoc, serverTimestamp } from
'firebase/firestore';
import { auth, db, googleProvider } from '../services/firebase';

export function useAuth() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [authorized, setAuthorized] = useState(null);

  useEffect(() => {
    return onAuthStateChanged(auth, async (u) => {
      if (u) {
        const authDoc = await getDoc(doc(db, 'authorizedUsers',
u.email));
        if (authDoc.exists()) {
          setAuthorized(true);
          await setDoc(doc(db, 'users', u.uid), {
            email: u.email,
            displayName: u.displayName,
            photoURL: u.photoURL,
            lastSeen: serverTimestamp()
          }, { merge: true });
        } else {
          setAuthorized(false);
        }
      } else {
        setAuthorized(null);
      }
      setUser(u);
      setLoading(false);
    });
  }, []);

  const login = () => signInWithPopup(auth, googleProvider);
  const logout = () => signOut(auth);

  return { user, loading, authorized, login, logout };
}
```

Phase 8 — Firestore data structure (reference)

These collections are **created automatically** when the first document is written. No need to pre-create.

```
authorizedUsers/{email}
  role      'user' | 'admin'
  addedAt   Timestamp
  addedBy   string

users/{uid}
  email, displayName, photoURL, lastSeen

chats/{chatId}
  ownerId, title, createdAt, updatedAt, messageCount

chats/{chatId}/messages/{msgId}    ← subcollection
  role      'user' | 'assistant'
  content, timestamp, hasImage?, tokensIn?, tokensOut?

testerLogs/{logId}
  type, message, uid, severity, meta, version, timestamp
```

Phase 9 — Security Rules (15 min, critical)

firestore.rules :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isSignedIn() {
      return request.auth != null;
    }

    function isAuthorized() {
      return isSignedIn() &&

exists (/databases/{database}/documents/authorizedUsers/{request.auth.token

    }

    function isAdmin() {
      return isSignedIn() &&

exists (/databases/{database}/documents/authorizedUsers/{request.auth.token

    &&

get (/databases/{database}/documents/authorizedUsers/{request.auth.token.e
    == 'admin';
  }

  function isOwner(uid) {
    return isSignedIn() && request.auth.uid == uid;
  }

  match /authorizedUsers/{email} {
    allow read: if isSignedIn();
    allow write: if isAdmin();
  }

  match /users/{uid} {
    allow read: if isOwner(uid) || isAdmin();
    allow write: if isOwner(uid) && isAuthorized();
  }

  match /chats/{chatId} {
    allow read: if isAuthorized() &&
      (resource.data.ownerId == request.auth.uid || isAdmin());
    allow create: if isAuthorized() &&
      request.resource.data.ownerId == request.auth.uid;
    allow update, delete: if isAuthorized() &&
      resource.data.ownerId == request.auth.uid;

    match /messages/{msgId} {
      allow read: if isAuthorized() && (

get (/databases/{database}/documents/chats/{chatId}).data.ownerId ==
request.auth.uid
      || isAdmin()
    );
    allow write: if false;
  }
}

  match /testerLogs/{logId} {
    allow read: if isAuthorized();
    allow write: if false;
  }
}
}
```

Deploy:

```
firebase deploy --only firestore:rules
```

Phase 10 — Cloud Function: chat with Gemini (25 min)

10.1 Initialize functions

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app\functions
npm init -y
npm install firebase-admin firebase-functions @google/generative-ai
```

macOS (Terminal)

```
cd ~/projects/my-app/functions
npm init -y
npm install firebase-admin firebase-functions @google/generative-ai
```

10.2 `functions/package.json`

Replace contents:

```
{
  "name": "functions",
  "description": "Cloud Functions for my-app",
  "version": "6.0.0",
  "main": "index.js",
  "engines": { "node": "20" },
  "scripts": {
    "lint": "echo 'no lint'",
    "serve": "firebase emulators:start --only functions",
    "deploy": "firebase deploy --only functions",
    "logs": "firebase functions:log"
  },
  "dependencies": {
    "@google/generative-ai": "^0.21.0",
    "firebase-admin": "^13.0.0",
    "firebase-functions": "^6.0.0"
  },
  "private": true
}
```

10.3 Get Gemini API key

1. `aistudio.google.com/app/apikey` → Create API key.
2. Choose project → Create.
3. Copy the key (`AIza...`).

10.4 Save as secret

```
firebase functions:secrets:set GEMINI_API_KEY
```

Paste the key when asked.

10.5 — VERSION A: Text only

`functions/index.js` :

```
// functions/index.js — VERSION A: text only
// _____
// Backend Cloud Functions. All AI logic and persistence
// requiring privileges lives here.
// _____

const { onCall, HttpsError } = require('firebase-functions/v2/https');
const { defineSecret } = require('firebase-functions/params');
const { GoogleGenerativeAI } = require('@google/generative-ai');
const admin = require('firebase-admin');

admin.initializeApp();
const db = admin.firestore();
const GEMINI_KEY = defineSecret('GEMINI_API_KEY');

const VERSION = '6.000.0';

// Helper: log to Code Tester
async function logTester(type, message, meta = {}, uid = null, severity
= 'info') {
```



```

    try {
      await db.collection('testerLogs').add({
        type, message, meta, uid, severity,
        version: VERSION,
        timestamp: admin.firestore.FieldValue.serverTimestamp()
      });
    } catch (err) {
      console.error('Error writing testerLog:', err);
    }
  }

  async function checkAuthorized(email) {
    const doc = await db.collection('authorizedUsers').doc(email).get();
    return doc.exists;
  }

  exports.chatWithGemini = onCall(
    {
      secrets: [GEMINI_KEY],
      region: 'europe-west1',
      timeoutSeconds: 540,
      memory: '512MiB',
      maxInstances: 10
    },
    async (request) => {
      const startTime = Date.now();

      if (!request.auth) {
        throw new HttpsError('unauthenticated', 'Authentication
required');
      }
      const uid = request.auth.uid;
      const email = request.auth.token.email;

      if (!await checkAuthorized(email)) {
        await logTester('error', 'Unauthorized email', { email }, uid,
'error');
        throw new HttpsError('permission-denied', 'Email not authorized');
      }

      const { chatId, userMessage } = request.data || {};
      if (!chatId || !userMessage || typeof userMessage !== 'string') {
        throw new HttpsError('invalid-argument', 'chatId and userMessage
required');
      }
      if (userMessage.length > 100000) {
        throw new HttpsError('invalid-argument', 'Message too long');
      }

      try {
        const chatRef = db.collection('chats').doc(chatId);
        const chatDoc = await chatRef.get();

        if (!chatDoc.exists) {
          await chatRef.set({
            ownerId: uid,
            title: userMessage.slice(0, 60),
            createdAt: admin.firestore.FieldValue.serverTimestamp(),
            updatedAt: admin.firestore.FieldValue.serverTimestamp(),
            messageCount: 0
          });
        } else if (chatDoc.data().ownerId !== uid) {
          throw new HttpsError('permission-denied', 'Not chat owner');
        }

        const messagesSnap = await chatRef
          .collection('messages')
          .orderBy('timestamp', 'asc')
          .get();

        const history = messagesSnap.docs.map(d => ({
          role: d.data().role === 'assistant' ? 'model' : 'user',
          parts: [{ text: d.data().content }]
        }));

        const genAI = new GoogleGenerativeAI(GEMINI_KEY.value());
        const model = genAI.getGenerativeModel({
          // Chosen model: gemini-2.5-pro (1M tokens context, multimodal)
          // Alternatives: 'gemini-2.5-flash', 'gemini-3-pro', 'gemini-3-
flash'
          model: 'gemini-2.5-pro',
          generationConfig: { maxOutputTokens: 8192, temperature: 0.7 }
          // systemInstruction: 'You are a helpful assistant...'
        });

```

```

const chat = model.startChat({ history });
const result = await chat.sendMessage(userMessage);
const responseText = result.response.text();
const usage = result.response.usageMetadata || {};

const batch = db.batch();
const userMsgRef = chatRef.collection('messages').doc();
batch.set(userMsgRef, {
  role: 'user',
  content: userMessage,
  timestamp: admin.firestore.FieldValue.serverTimestamp()
});

const assistantMsgRef = chatRef.collection('messages').doc();
batch.set(assistantMsgRef, {
  role: 'assistant',
  content: responseText,
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
  tokensIn: usage.promptTokenCount || 0,
  tokensOut: usage.candidatesTokenCount || 0
});

batch.update(chatRef, {
  updatedAt: admin.firestore.FieldValue.serverTimestamp(),
  messageCount: admin.firestore.FieldValue.increment(2)
});

await batch.commit();

await logTester('gemini-call', `Chat ${chatId.slice(0, 8)}
(${history.length} msgs)`, {
  chatId,
  historyLen: history.length,
  tokensIn: usage.promptTokenCount,
  tokensOut: usage.candidatesTokenCount,
  ms: Date.now() - startTime
}, uid, 'info');

return {
  response: responseText,
  usage: { tokensIn: usage.promptTokenCount, tokensOut:
usage.candidatesTokenCount }
};

} catch (err) {
  await logTester('error', err.message || 'unknown', { chatId,
stack: err.stack?.slice(0, 500) }, uid, 'error');
  if (err instanceof HttpsError) throw err;
  throw new HttpsError('internal', err.message || 'Internal error');
}
}
);

exports.healthCheck = onCall(
  { region: 'europe-west1' },
  async () => ({ ok: true, version: VERSION, timestamp: Date.now() })
);

```

10.6 — VERSION B: Text + images

Replace the “Call Gemini” portion (step 4 inside the function) with:

```

const genAI = new GoogleGenerativeAI(GEMINI_KEY.value());
const model = genAI.getGenerativeModel({
  model: 'gemini-2.5-pro',
  generationConfig: { maxOutputTokens: 8192, temperature: 0.7 }
});
const chat = model.startChat({ history });

const parts = [{ text: userMessage }];
if (request.data.imageBase64 && request.data.imageMimeType) {
  parts.push({
    inlineData: {
      mimeType: request.data.imageMimeType,
      data: request.data.imageBase64
    }
  });
}

const result = await chat.sendMessage(parts);
const responseText = result.response.text();
const usage = result.response.usageMetadata || {};

```

And in step 5, add `hasImage` :

```
batch.set(userMsgRef, {
  role: 'user',
  content: userMessage,
  hasImage: !!request.data.imageBase64,
  timestamp: admin.firestore.FieldValue.serverTimestamp()
});
```

10.7 Deploy

```
cd ..
firebase deploy --only functions
```

📖 Phase 11 — Frontend persistent chat (25 min)

🧠 **Philosophy reminder:** each component in its own file. To add anything new, **create a new component**, don't inflate `Chat.jsx` .

11.1 `web/src/components/Chat.jsx`

```
// web/src/components/Chat.jsx
// -----
// Single component responsible for the chat: reads
// messages in real-time, sends new ones, handles
// optional images.
// -----
import { useState, useEffect, useRef } from 'react';
import { collection, query, orderBy, onSnapshot } from
'firebase/firestore';
import { httpsCallable } from 'firebase/functions';
import { db, functions } from '../services/firebase';

export function Chat({ user, chatId }) {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const [imageBase64, setImageBase64] = useState(null);
  const [imageMime, setImageMime] = useState(null);
  const endRef = useRef(null);
  const fileRef = useRef(null);

  useEffect(() => {
    if (!chatId) { setMessages([]); return; }
    const q = query(
      collection(db, 'chats', chatId, 'messages'),
      orderBy('timestamp', 'asc')
    );
    return onSnapshot(q, (snap) => {
      setMessages(snap.docs.map(d => ({ id: d.id, ...d.data() })));
      setTimeout(() => endRef.current?.scrollIntoView({ behavior:
'smooth' }), 50);
    });
  }, [chatId]);

  const handleImage = (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    if (file.size > 4 * 1024 * 1024) {
      alert('Image too large (max 4MB)');
      return;
    }
    const reader = new FileReader();
    reader.onload = () => {
      const base64 = reader.result.split(',')[1];
      setImageBase64(base64);
      setImageMime(file.type);
    };
    reader.readAsDataURL(file);
  };

  const send = async () => {
```

```

    if ((!input.trim() && !imageBase64) || loading) return;
    const msg = input.trim() || 'What do you see in this image?';
    const img = imageBase64;
    const mime = imageMime;
    setInput('');
    setImageBase64(null);
    setImageMime(null);
    if (fileRef.current) fileRef.current.value = '';
    setLoading(true);
    try {
      const fn = httpsCallable(functions, 'chatWithGemini');
      await fn({
        chatId,
        userId,
        userMessage: msg,
        imageBase64: img,
        imageMimeType: mime
      });
    } catch (err) {
      alert('Error: ' + err.message);
      setInput(msg);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ display: 'flex', flexDirection: 'column', height:
'100vh', background: '#111' }}>
      <div style={{ flex: 1, overflowY: 'auto', padding: 16 }}>
        {messages.length === 0 && (
          <div style={{ color: '#666', textAlign: 'center', marginTop:
80 }}>
            Start the conversation
          </div>
        )}
        {messages.map(m => (
          <div key={m.id} style={{
            margin: '8px 0',
            display: 'flex',
            justify-content: m.role === 'user' ? 'flex-end' : 'flex-
start'
          }}>
            <div style={{
              padding: '10px 14px',
              borderRadius: 12,
              background: m.role === 'user' ? '#0066cc' : '#2a2a2a',
              color: '#fff',
              maxWidth: '70%',
              whiteSpace: 'pre-wrap',
              wordBreak: 'break-word'
            }}>
              {m.hasImage && <div style={{ fontSize: 11, opacity: 0.7,
marginBottom: 4 }}>🖼️ with image</div>
              {m.content}
            </div>
          </div>
        ))}
        {loading && (
          <div style={{ color: '#888', fontStyle: 'italic', padding: 8
}}>
            Gemini is typing...
          </div>
        )}
        <div ref={endRef} />
      </div>

      {imageBase64 && (
        <div style={{ padding: 8, background: '#1a1a1a', color: '#fff',
fontSize: 12 }}>
          🖼️ Image ready to send
          <button
            onClick={() => { setImageBase64(null); setImageMime(null);
if (fileRef.current) fileRef.current.value = ''; }}
            style={{ marginLeft: 8, background: 'transparent', color:
'#fa0', border: 'none', cursor: 'pointer' }}>
            (remove)
          </button>
        </div>
      )}

      <div style={{ padding: 16, borderTop: '1px solid #333', display:
'flex', gap: 8, background: '#1a1a1a' }}>
        <input
          ref={fileRef}
          type="file"

```

```

        accept="image/*"
        onChange={handleImage}
        style={{ display: 'none' }}
        id="imgInput"
      />
    <label
      htmlFor="imgInput"
      style={{
        padding: '10px 14px', background: '#333', color: '#fff',
        borderRadius: 8, cursor: 'pointer', display: 'flex',
        alignItems: 'center'
      }}>
    </label>
    <input
      value={input}
      onChange={e => setInput(e.target.value)}
      onKeyDown={e => e.key === 'Enter' && !e.shiftKey && send()}
      disabled={loading}
      placeholder="Type a message..."
      style={{
        flex: 1, padding: 10, background: '#222', color: '#fff',
        border: '1px solid #444', borderRadius: 8
      }}
    />
    <button
      onClick={send}
      disabled={loading || (!input.trim() && !imageBase64)}
      style={{
        padding: '10px 20px', background: '#0066cc', color: '#fff',
        border: 'none', borderRadius: 8, cursor: loading ? 'wait' :
        'pointer'
      }}
      {loading ? '...' : 'Send'}
    </button>
  </div>
</div>
);
}

```

11.2 web/src/components/CodeTester.jsx

🔔 **Reminder:** this component is the conscience of the app. Every important backend operation should call `logTester(...)` so it shows up here.

```

// web/src/components/CodeTester.jsx
// -----
// Floating console at bottom-right showing logs that
// Cloud Functions write to 'testerLogs'. Updates in real
// time. Also a thermometer of architectural health.
// -----
import { useState, useEffect } from 'react';
import { collection, query, orderBy, limit, onSnapshot } from
'firebase/firestore';
import { db } from '../services/firebase';

const SEVERITY_COLOR = { info: '#0f0', warn: '#fa0', error: '#f33' };

export function CodeTester() {
  const [logs, setLogs] = useState([]);
  const [collapsed, setCollapsed] = useState(false);
  const [filter, setFilter] = useState('all');

  useEffect(() => {
    const q = query(
      collection(db, 'testerLogs'),
      orderBy('timestamp', 'desc'),
      limit(100)
    );
    return onSnapshot(
      q,
      (snap) => setLogs(snap.docs.map(d => ({ id: d.id, ...d.data()
}))),
      (err) => console.error('CodeTester listener error:', err)
    );
  }, []);

  const visible = filter === 'all' ? logs : logs.filter(l => l.severity
=== filter);

  return (

```

```

<div style={{
  position: 'fixed',
  bottom: 0,
  right: 0,
  width: collapsed ? 180 : 520,
  height: collapsed ? 28 : 340,
  background: '#000',
  color: '#0f0',
  fontFamily: 'Menlo, Consolas, monospace',
  fontSize: 11,
  borderTop: '1px solid #0f0',
  borderLeft: '1px solid #0f0',
  zIndex: 9999,
  display: 'flex',
  flexDirection: 'column'
}}>
  <div
    onClick={() => setCollapsed(!collapsed)}
    style={{
      padding: '4px 8px',
      borderBottom: collapsed ? 'none' : '1px solid #0f0',
      cursor: 'pointer',
      userSelect: 'none',
      display: 'flex',
      justifyContent: 'space-between',
      alignItems: 'center'
    }}>
    <span>{collapsed ? '▸' : '▾'} CODE TESTER · {visible.length}</span>
    {!collapsed && (
      <select
        value={filter}
        onChange={e => { e.stopPropagation();
setFilter(e.target.value); }}
        onClick={e => e.stopPropagation()}
        style={{ background: '#000', color: '#0f0', border: '1px
solid #0f0', fontSize: 10 }}>
        <option value="all">all</option>
        <option value="info">info</option>
        <option value="warn">warn</option>
        <option value="error">error</option>
      </select>
    )}
  </div>
  {!collapsed && (
    <div style={{ flex: 1, overflowY: 'auto', padding: 6 }}>
      {visible.length === 0 && <div style={{ color: '#444' }}>no
logs</div>}
      {visible.map(log => {
        const t = log.timestamp?.toDate?.()?.toISOString().slice(11,
19) || '...';
        const c = SEVERITY_COLOR[log.severity] || '#0f0';
        return (
          <div key={log.id} style={{ color: c, marginBottom: 2,
lineHeight: 1.4 }}>
            <span style={{ color: '#666' }}>[{t}]</span>{' '}
            <span style={{ color: '#888' }}>[{log.type}]</span>{' '}
            {log.message}
            {log.meta && Object.keys(log.meta).length > 0 && (
              <span style={{ color: '#666', marginLeft: 6 }}>
                {JSON.stringify(log.meta).slice(0, 120)}
              </span>
            )}
          </div>
        );
      })}
    </div>
  )}
</div>
);
}

```

11.3 web/src/App.jsx

🕒 **App.jsx** only routes between states and composes other components. All real logic lives elsewhere.

```

// web/src/App.jsx
// _____
// Root component. Only decides which screen to show.

```

```

// Chat, auth, log, admin logic lives in their own files.
// -----
import { useState, useEffect } from 'react';
import { collection, query, where, orderBy, onSnapshot, addDoc,
serverTimestamp } from 'firebase/firestore';
import { db } from '../services/firebase';
import { useAuth } from '../hooks/useAuth';
import { Chat } from '../components/Chat';
import { CodeTester } from '../components/CodeTester';
import { VER, APP_NAME } from '../constants';

export default function App() {
  const { user, loading, authorized, login, logout } = useAuth();
  const [chats, setChats] = useState([]);
  const [activeChatId, setActiveChatId] = useState(null);

  useEffect(() => {
    if (!user || !authorized) { setChats([]); return; }
    const q = query(
      collection(db, 'chats'),
      where('ownerId', '==', user.uid),
      orderBy('updatedAt', 'desc')
    );
    return onSnapshot(q, (snap) => {
      setChats(snap.docs.map(d => ({ id: d.id, ...d.data() })));
    });
  }, [user, authorized]);

  if (loading) return <div style={{ padding: 32, color: '#fff' }}>Loading...</div>;

  if (!user) return (
    <div style={{
      display: 'flex', flexDirection: 'column', alignItems: 'center',
      justifyContent: 'center',
      height: '100vh', background: '#111', color: '#fff', gap: 24
    }}>
      <h1>{APP_NAME}</h1>
      <button
        onClick={login}
        style={{
          padding: '14px 28px', background: '#fff', color: '#111',
          border: 'none', borderRadius: 8, fontSize: 16, cursor:
'pointer'
        }}>
        Login with Google
      </button>
      <small style={{ color: '#666' }}>{VER}</small>
    </div>
  );

  if (!authorized) return (
    <div style={{
      display: 'flex', flexDirection: 'column', alignItems: 'center',
      justifyContent: 'center',
      height: '100vh', background: '#1a0000', color: '#fff', gap: 16,
      padding: 32, textAlign: 'center'
    }}>
      <h2>⚠ Access not authorized</h2>
      <p>Email <strong>{user.email}</strong> is not on the authorized
users list.</p>
      <p style={{ color: '#888' }}>If you think you should be, ask the
app owner.</p>
      <button onClick={logout}>Logout</button>
    </div>
  );

  const newChat = async () => {
    const ref = await addDoc(collection(db, 'chats'), {
      ownerId: user.uid,
      title: 'New chat',
      createdAt: serverTimestamp(),
      updatedAt: serverTimestamp(),
      messageCount: 0
    });
    setActiveChatId(ref.id);
  };

  return (
    <div style={{ display: 'flex', height: '100vh', background:
'#0a0a0a', color: '#fff' }}>
      <aside style={{
        width: 240, borderRight: '1px solid #333', padding: 12,
        display: 'flex', flexDirection: 'column', gap: 8
      }}>

```

```

        <div style={{ fontSize: 12, color: '#888' }}>{user.email}</div>
        <button onClick={newChat} style={{ padding: 8 }}>+ New
      chat</button>
      <div style={{ flex: 1, overflowY: 'auto' }}>
        {chats.map(c => (
          <div
            key={c.id}
            onClick={() => setActiveChatId(c.id)}
            style={{
              padding: 8, cursor: 'pointer', borderRadius: 4,
              background: c.id === activeChatId ? '#0066cc' :
'transparent',
              marginBottom: 2, fontSize: 13
            }}>
              {c.title}
            </div>
          )
        )}
      </div>
      <button onClick={logout} style={{ padding: 6, fontSize: 12
    }}>Logout</button>
    <small style={{ color: '#444', textAlign: 'center' }}>{VER}
  </small>
  </aside>
  <main style={{ flex: 1 }}>
    {activeChatId
      ? <Chat user={user} chatId={activeChatId} />
      : <div style={{ padding: 32, color: '#666' }}>Select or create
a chat</div>}
  </main>
  <CodeTester />
</div>
);
}

```

11.4 `web/index.html`

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <meta name="version" content="v6.000" />
    <title>My App</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

11.5 Test locally

```

cd web
npm run dev

```

Open `http://localhost:5173` .

🔗 Phase 12 — Admin frontend (15 min)

12.1 Initialize `admin/`

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app\admin
npm create vite@latest . -- --template react
npm install
npm install firebase
```

macOS (Terminal)

```
cd ~/projects/my-app/admin
npm create vite@latest . -- --template react
npm install
npm install firebase
```

12.2 Copy shared files

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app\admin
mkdir src\services -Force
mkdir src\hooks -Force
mkdir src\components -Force
Copy-Item ../web/src/services/firebase.js src/services\
Copy-Item ../web/src/hooks/useAuth.js src/hooks\
Copy-Item ../web/src/components/CodeTester.jsx src/components\
Copy-Item ../web/src/constants.js src\
```

macOS (Terminal)

```
cd ~/projects/my-app/admin
mkdir -p src/services src/hooks src/components
cp ../web/src/services/firebase.js src/services/
cp ../web/src/hooks/useAuth.js src/hooks/
cp ../web/src/components/CodeTester.jsx src/components/
cp ../web/src/constants.js src/
```

Edit `admin/src/constants.js` :

```
export const APP_NAME = "My App — Admin";
```

12.3 `admin/src/components/AdminPanel.jsx`

```
// admin/src/components/AdminPanel.jsx
// _____
// Authorized users management panel.
// Only visible if email is in authorizedUsers with role: admin.
// _____
import { useState, useEffect } from 'react';
import {
  collection, doc, query, orderBy, onSnapshot,
  setDoc, deleteDoc, serverTimestamp
} from 'firebase/firestore';
import { db } from '../services/firebase';

export function AdminPanel({ user, logout }) {
  const [authorizedList, setAuthorizedList] = useState([]);
  const [users, setUsers] = useState([]);
  const [newEmail, setNewEmail] = useState('');
  const [newRole, setNewRole] = useState('user');

  useEffect(() => {
    const unsubA = onSnapshot(collection(db, 'authorizedUsers'), (snap) => {
      setAuthorizedList(snap.docs.map(d => ({ id: d.id, ...d.data() })));
    });
    const qU = query(collection(db, 'users'), orderBy('lastSeen', 'desc'));
    const unsubU = onSnapshot(qU, (snap) => {
      setUsers(snap.docs.map(d => ({ id: d.id, ...d.data() })));
    });
    return () => { unsubA(); unsubU(); };
  });
}
```



```

{a.addedBy || '-'}</td>
      <td style={{ padding: 8, textAlign: 'right' }}>
        <button onClick={() => toggleAdmin(a.id, a.role)}
style={{ marginRight: 8 }}>
          {a.role === 'admin' ? '↓ user' : '↑ admin'}
        </button>
        <button onClick={() => removeUser(a.id)} style={{
background: '#600' }}>x</button>
      </td>
    </tr>
  )}
</tbody>
</table>
</section>

<section>
  <h2>User activity ({users.length})</h2>
  <table style={{ width: '100%', borderCollapse: 'collapse' }}>
    <thead>
      <tr style={{ borderBottom: '1px solid #444' }}>
        <th align="left" style={{ padding: 8 }}>Email</th>
        <th align="left" style={{ padding: 8 }}>Name</th>
        <th align="left" style={{ padding: 8 }}>Last seen</th>
      </tr>
    </thead>
    <tbody>
      {users.map(u => (
        <tr key={u.id} style={{ borderBottom: '1px solid #222' }}>
          <td style={{ padding: 8 }}>{u.email}</td>
          <td style={{ padding: 8 }}>{u.displayName}</td>
          <td style={{ padding: 8, fontSize: 12, color: '#888' }}>
            {u.lastSeen?.toDate?.()?.toLocaleString() || '-'}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</section>
</div>
);
}

```

12.4 admin/src/App.jsx

```
// admin/src/App.jsx
import { useAuth } from '../hooks/useAuth';
import { AdminPanel } from '../components/AdminPanel';
import { CodeTester } from '../components/CodeTester';
import { VER } from '../constants';
import { useState, useEffect } from 'react';
import { doc, getDoc } from 'firebase/firestore';
import { db } from '../services/firebase';

export default function App() {
  const { user, loading, login, logout } = useAuth();
  const [isAdmin, setIsAdmin] = useState(null);

  useEffect(() => {
    if (!user) { setIsAdmin(false); return; }
    getDoc(doc(db, 'authorizedUsers', user.email)).then(snap => {
      setIsAdmin(snap.exists() && snap.data().role === 'admin');
    });
  }, [user]);

  if (loading || isAdmin === null) {
    return <div style={{ padding: 32, color: '#fff' }}>Loading...</div>;
  }

  if (!user) return (
    <div style={{
      display: 'flex', flexDirection: 'column', alignItems: 'center',
      justifyContent: 'center',
      height: '100vh', background: '#111', color: '#fff', gap: 24
    }}>
      <h1>Admin · My App</h1>
      <button onClick={login} style={{ padding: '14px 28px', fontSize:
16 }}>
        Login with Google
      </button>
      <small style={{ color: '#666' }}>{VER}</small>
    </div>
  );

  if (!isAdmin) return (
    <div style={{ padding: 32, color: '#fff', background: '#1a0000',
minHeight: '100vh' }}>
      <h2>⚠ Not authorized</h2>
      <p>{user.email} is not an admin.</p>
      <button onClick={logout}>Logout</button>
    </div>
  );

  return (
    <>
      <AdminPanel user={user} logout={logout} />
      <CodeTester />
    </>
  );
}
```

12.5 admin/index.html

```
<meta name="version" content="v6.000" />
<title>My App – Admin</title>
```

👤 Phase 13 — Add the first admin (1 min, only first time)

The only step you **CANNOT** do from the UI (rules say “only admins can add admins” → must crack the egg first).

1. Firebase Console → **Firestore Database**.
2. **Start collection** → ID: `authorizedUsers`.
3. **Document ID**: your exact email (e.g., `admin-name@gmail.com`).
4. Fields:
 - `role` (string) → `admin`
 - `addedAt` (timestamp) → click clock for current time

- `addedBy` (string) → `manual`

5. Save.

Phase 14 — Build and deploy (10 min)

 **Claude Code's moment of glory.** Tell it: *"Build both frontends (web/ and admin/) and deploy to Firebase."* One session, all done.

14.1 Manual build (if you prefer)

Windows (PowerShell)

```
cd $env:USERPROFILE\projects\my-app\web
npm run build
cd $env:USERPROFILE\projects\my-app\admin
npm run build
cd $env:USERPROFILE\projects\my-app
```

macOS (Terminal)

```
cd ~/projects/my-app/web && npm run build
cd ~/projects/my-app/admin && npm run build
cd ~/projects/my-app
```

14.2 Full deploy

```
firebase deploy
```

14.3 Verify

- `https://my-app.web.app` → login → chat with Gemini.
- `https://my-app-admin.web.app` → admin panel.

Phase 15 — `CLAUDE.md` for Claude Code

Particularly important if you use Claude Code, as it provides the context Claude needs to work efficiently without burning many tokens.

At the repo root, `CLAUDE.md` :

```

# CLAUDE.md — My App

## Code philosophy (READ BEFORE TOUCHING ANYTHING)

**One file, one feature.** Never inflate App.jsx or other components.
Add new features to their own files inside `web/src/components/` or
`web/src/hooks/`.

Every important backend operation must call `logTester(...)` so it
appears in the Code Tester.

## Workflow

- **Claude chat (claude.ai)** designs and writes new code. Generates
  zips.
- **You (Claude Code)** execute: unzip, build, deploy, git. NO feature
  dev.
- **Code Tester** reads testerLogs/ autonomously on the frontend.
- The human deploys manually for visual verification.

## Stack

- Frontend: Vite + React (JavaScript)
- Backend: Cloud Functions Node.js 20
- Firestore: region eur3
- Functions: region europe-west1
- Auth: Google only
- AI Model: Gemini 2.5 Pro (1M tokens context, multimodal)

## Structure

- `web/` — user frontend
- `admin/` — admin frontend
- `functions/` — Cloud Functions
- `firestore.rules` — security rules
- `firebase.json` — global Firebase config

## Versioning — 7 locations to sync on every bump

Format: `v6000` (constants.js, html), `6.000.0` (functions/index.js),
`6.0.0` (functions/package.json).

1. `web/src/constants.js` → `export const VER = "v6000";`
2. `web/package.json` → `"version": "6.000.0"`
3. `web/index.html` → `

```

📦 Phase 16 — Final commit v6.000

```

git add .
git commit -m "v6.000 - App live: auth, chat, admin, code tester"
git push

```

✓ Phase 17 — Final checklist

Step	Status
Private GitHub repo created	☐
Firebase project on Blaze	☐
Budget alert configured (€5/\$5 monthly)	☐
Google Auth enabled	☐
Firestore created in <code>eur3</code>	☐
Multi-site hosting (web + admin)	☐
Security rules deployed	☐
Cloud Function <code>chatWithGemini</code> deployed with secret	☐
Web frontend: login + chat + Code Tester	☐
Admin frontend: gating by <code>role: admin</code>	☐
First admin user added manually to Firestore	☐
7 version locations consistent	☐
<code>my-app.web.app</code> accessible from mobile	☐
<code>my-app-admin.web.app</code> only allows admins	☐
Code Tester shows real-time logs	☐
Each component in its own file (no monsters)	☐
<code>CLAUDE.md</code> at repo root for Claude Code	☐

🔧 Common issues & solutions

“Permission denied” writing to Firestore

Check `firestore.rules`. Likely the user is not in `authorizedUsers/`. Add them via Firebase Console or admin panel.

Cloud Function timeout

Default 60s. Already set to 540s in code. If Gemini takes longer (very large history), switch to `gemini-2.5-flash` (faster).

“GEMINI_API_KEY is undefined”

Did you run `firebase functions:secrets:set GEMINI_API_KEY` ? And redeploy after?

```
firebase deploy --only functions
```

“Firebase: Error (auth/popup-closed-by-user)”

Someone closed the popup. Not significant.

Vite build fails with “Could not resolve ...”

Missing `npm install` in that folder. `web/` and `admin/` are separate npm projects.

Windows: “scripts disabled on this system”

Repeat Phase 0:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

macOS: `gh: command not found`

Did you run the two final Homebrew lines? Verify with `which brew`.

Firebase deploy slow

First deploy takes 5 minutes (enabling APIs). If longer, `Ctrl+C` and `firebase deploy --debug`.

“Site not found” on deploy

Site IDs in `.firebaserc` don’t match Firebase Console → Hosting.

Code Tester stays empty

Check browser console (F12). Likely Firestore rules denying read because your email isn’t in `authorizedUsers`.

Image not working

1. Without `data:image/...,` prefix.
2. Correct `contentType`.
3. Under 4 MB.
4. Multimodal model.

My `App.jsx` is growing too much

Sign to extract pieces. **Never let `App.jsx` exceed ~150 lines.**

Burning too many Pro plan messages

Re-read “The Claude workflow”. You may be doing in chat what belongs in Claude Code (installs, builds, deploys). Chat for thinking, Code for executing.

Next steps

When you want to specialize your app:

1. Add `systemInstruction` (Phase 10.5):

```
const model = genAI.getGenerativeModel({
  model: 'gemini-2.5-pro',
  systemInstruction: 'You are a Mediterranean cooking expert...'
});
```

2. Customize the interface (Phase 11.1) — **remember: new components in new files.**
3. Add specific collections (`userPreferences/{uid}`, etc.) and have the Cloud Function load them into context.

This template, as is, **is already a fully functional app**. You can start using it today.

Additional resources

- **Firebase docs:** <https://firebase.google.com/docs>
- **Gemini API docs:** <https://ai.google.dev/gemini-api/docs>

- **Vite docs:** <https://vitejs.dev/>
 - **React docs:** <https://react.dev/>
 - **Claude Code docs:** <https://docs.claude.com/en/docs/claude-code>
 - **Firestore Security Rules:**
<https://firebase.google.com/docs/firestore/security/get-started>
-

End of manual. If you've reached this point with everything working, you now have your own private app, with massive AI on the internet, accessible from your phone, controlled by you, for a few euros a month at most.

And the best part: you've built it **without exhausting your Claude Pro plan**, thanks to the chat → Code → Tester workflow. Each tool in its place, each piece in its file.